



Scalable Distributed Inference Platform

A powerful multi-node inference system enabling distributed AI/ML workloads with automatic discovery, telemetry, flexible result publishing, and multi-engine support.



Platform Highlights

Core Features

- Multi-engine support (YOLO / ONNX / Geti / Custom)
 - Cross-node auto-discovery
 - Real-time telemetry (CPU, RAM, GPU, network, inference metrics)
 - Multiple output destinations (MQTT, Webhook, OPC-UA, Serial, ROS2, ZeroMQ, Folder...)
 - REST API for full remote control
 - Built-in rate limiting for stable result streaming
-



Supported Inference Engines

- **Ultralytics YOLO** (v8, v11...)
 - **Intel Geti**
 - **ONNX Runtime** (CPU / OpenVINO / CUDA GPU)
 - **Pass-through**
 - **Custom engines** using BaseInferenceEngine
-



Result Destinations

- MQTT

- HTTP Webhook
 - Serial(USB / RS-232)
 - OPC-UA
 - ROS2
 - ZeroMQ
 - Folder storage
 - Roboflow
 - Geti
 - Custom plugins
-



Requirements

- Python 3.10+
 - Linux or Windows
 - Optional: CUDA-enabled GPU
 - Optional: MQTT broker
-



Installation

Quick Setup

```
git clone https://github.com/olkham/inference_node.git
cd inference_node
```

```
# Windows
setup.bat
```

```
# Linux/Mac
chmod +x setup.sh
./setup.sh
```

Manual Installation

```
pip install -r requirements.txt
pip install ultralytics torch torchvision geti-sdk

pip install onnxruntime
pip install onnxruntime-openvino
pip install "onnxruntime-gpu[cuda12,cudnn]"

pip install nvidia-ml-py>=12.0.0
pip install pyserial>=3.5
```



Quick Start

Start Node (Python)

```
from InferenceNode import InferenceNode

node = InferenceNode("MyNode", port=5555)
node.start(enable_discovery=True, enable_telemetry=True)
```

CLI Modes

```
python main.py
python main.py --production
python main.py --port 8080 --name "ProdNode"
```



Result Publishing

```
from ResultPublisher import ResultPublisher, ResultDestination

rp = ResultPublisher()

mqtt = ResultDestination("mqtt")
```

```
mqtt.configure(server="localhost", topic="infer/results", rate_limit=1.0)
rp.add(mqtt)
```



REST API Endpoints

```
POST /api/engine/load
POST /api/engine/upload
POST /api/inference
POST /api/telemetry/start
POST /api/telemetry/stop
POST /api/publisher/configure
```



Project Structure

```
inference_node/
├─ InferenceEngine/
├─ InferenceNode/
├─ ResultPublisher/
├─ main.py
├─ docker-compose.yml
├─ Dockerfile
└─ requirements.txt
```

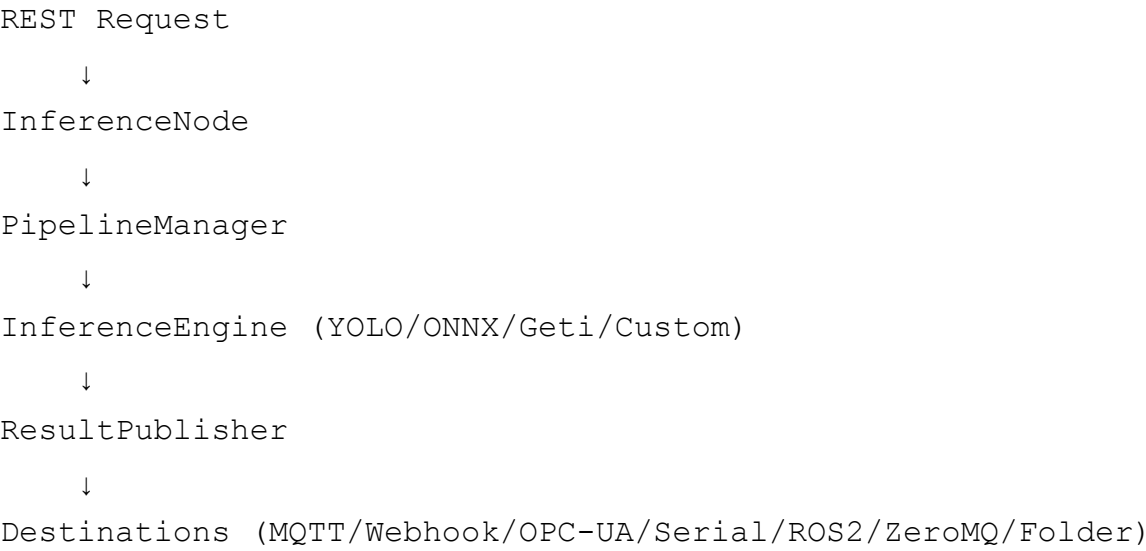


Example Telemetry Output

```
{
  "node_id": "abcd-1234",
  "timestamp": "2025-07-28T10:30:00Z",
  "cpu": 45.2,
  "memory": 67.3,
  "gpu_available": true
}
```



Python Object Interaction Pipeline



Internal Responsibilities

Component	Role
InferenceNode	Orchestration, API, telemetry, discovery
PipelineManager	Routing, pipeline selection, engine management
InferenceEngine	ML inference lifecycle (load, preprocess, infer, postprocess)
ResultPublisher	Dispatch results to multiple destinations
Telemetry	System performance reporting
Destinations	Output plugins



Flow (Detailed Lifecycle)

1. **Request arrives** → InferenceNode
2. **Node selects pipeline** → PipelineManager
3. **Engine does inference** → InferenceEngine
4. **Results broadcasted** → ResultPublisher
5. **Telemetry continuously emitted** → MQTT



Running as a Linux Service (systemd)

1. Create service file

```
sudo nano /etc/systemd/system/infernode.service
```

Add:

```
[Unit]
Description=InferNode AI Inference Service
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/infernode
ExecStart=/usr/bin/python3 main.py --production
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

2. Copy project

```
sudo mkdir -p /opt/infernode
sudo cp -r /path/to/inference_node/* /opt/infernode
```

3. Reload and enable service

```
sudo systemctl daemon-reload
sudo systemctl enable infernode
sudo systemctl start infernode
```

4. View logs

```
sudo journalctl -u infernode -f
```



UML Enterprise Multi-Node Architecture

flowchart LR

```
subgraph NodeA[Inference Node A]
    A1[InferenceNode] --> B1[PipelineManager]
    B1 --> C1[InferenceEngine]
    C1 --> D1[ResultPublisher]
    A1 --> T1[Telemetry]
end

subgraph NodeB[Inference Node B]
    A2[InferenceNode] --> B2[PipelineManager]
    B2 --> C2[InferenceEngine]
    C2 --> D2[ResultPublisher]
    A2 --> T2[Telemetry]
end

A1 <--> A2

T1 --> MQTT[(MQTT Broker)]
T2 --> MQTT

D1 --> EXT[(External Systems)]
D2 --> EXT
```



Roadmap

- ✓ Web UI
- ✓ Telemetry streaming
- ✓ Multi-engine support
- ✓ Dockerized deployment
- ☐ Load balancing
- ☐ A/B testing for models
- ☐ Visual pipeline builder

